

Aaditya Dawar

1024150093

2013

## Lab Assignment 5

1. Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

```
#include <iostream>
```

```
using namespace std;
```

```
class Base {
```

```
public:
```

```
    void showBase() {
```

```
        cout << "This is Base Class" << endl;
```

```
    }
```

```
};
```

```
class Derived : public Base {
```

```
public:
```

```
    void showDerived() {
```

```
        cout << "This is Derived Class" << endl;
```

```
    }
```

```
};
```

```
int main() {
```

```
    Base b;
```

```
    Derived d;
```

```
    b.showBase();
```

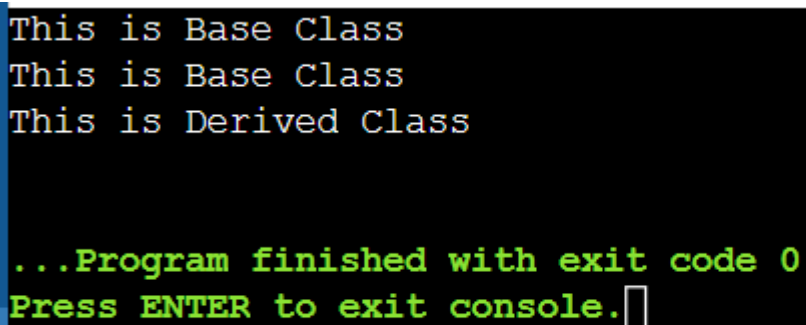
```
    d.showBase();
```

```
    d.showDerived();
```

```
    return 0;
```

```
}
```

Code-



```
This is Base Class  
This is Base Class  
This is Derived Class  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

2. Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

```
#include <iostream>  
using namespace std;
```

```
class Base {  
protected:  
    int value;
```

```
};

class Derived : public Base {
public:
    void input() {
        cin >> value;
    }

    void display() {
        cout << "Value is: " << value << endl;
    }
};

int main() {
    Derived d;

    cout << "Enter value: ";
    d.input();

    d.display();

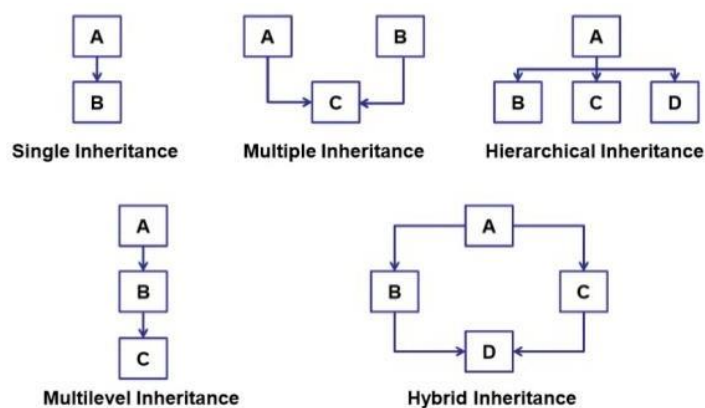
    return 0;
}
```

OUTPUT-

```
Enter value: Value is: 5

...Program finished with exit code 0
Press ENTER to exit console.
```

3. Various types of inheritances are as shown below. Write small C++ codes for each inheritance type.



Single inheritance-

```

#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "Class A" << endl;
    }
};

class B : public A {
public:
    void showB() {
        cout << "Class B" << endl;
    }
};

int main() {
    B obj;
    obj.showA();
    obj.showB();
    return 0;
}

```

Multiple inheritance-

```

#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "Class A" << endl;
    }
};

class B {
public:
    void showB() {
        cout << "Class B" << endl;
    }
};

class C : public A, public B {
};

int main() {
    C obj;
    obj.showA();
    obj.showB();
    return 0;
}

```

```
}
```

Hierarchical inheritance-

```
#include <iostream>
```

```
using namespace std;
```

```
class A {
```

```
public:
```

```
    void showA() {
```

```
        cout << "Class A" << endl;
```

```
    }
```

```
};
```

```
class B : public A {};
```

```
class C : public A {};
```

```
int main() {
```

```
    B obj1;
```

```
    C obj2;
```

```
    obj1.showA();
```

```
    obj2.showA();
```

```
    return 0;
```

```
}
```

Multilevel inheritance-

```
#include <iostream>
```

```
using namespace std;
```

```
class A {
```

```
public:
```

```
    void showA() {
```

```
        cout << "Class A" << endl;
```

```
    }
```

```
};
```

```
class B : public A {};
```

```
class C : public A {};
```

```
int main() {
```

```
    B obj1;
```

```
    C obj2;
```

```
    obj1.showA();
```

```
    obj2.showA();
```

```
    return 0;
```

```
}
```

Hybrid inheritance-

```
#include <iostream>
```

```

using namespace std;

class A {
public:
    void showA() {
        cout << "Class A" << endl;
    }
};

class B : public A {};
class C : public A {};

class D : public B, public C {};

int main() {
    D obj;
    obj.B::showA();
    obj.C::showA();
    return 0;
}

```

4. How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

Code-

```

#include <iostream>
using namespace std;

class Base {
public:
    Base() {
        cout << "Base Constructor" << endl;
    }

    ~Base() {
        cout << "Base Destructor" << endl;
    }
};

class Derived : public Base {
public:
    Derived() {
        cout << "Derived Constructor" << endl;
    }
}

```

```

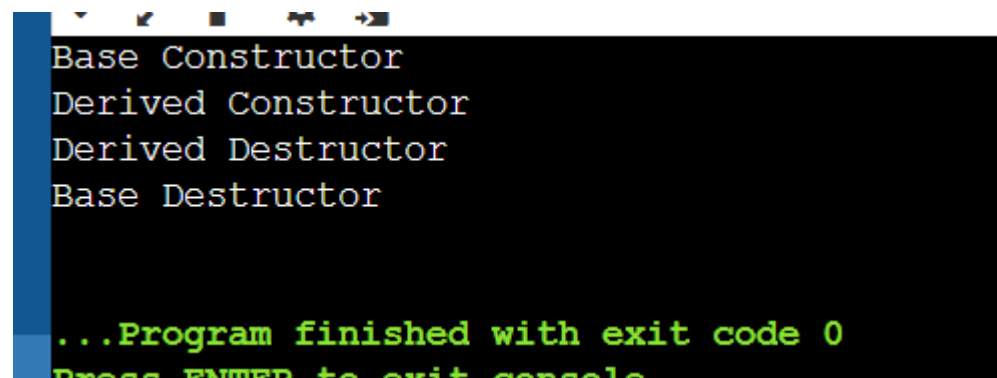
~Derived() {
    cout << "Derived Destructor" << endl;
}
};

```

```

int main() {
    Derived obj;
    return 0;
}

```



```

Base Constructor
Derived Constructor
Derived Destructor
Base Destructor

...Program finished with exit code 0
Press ENTER to exit console

```

5. Implement a base class Book with attributes title, author, and price. Then, create a derived class Textbook that inherits from Book and adds a new attribute subject. Demonstrate how single inheritance is used to manage the data for general books and textbooks.

```
#include <iostream>
```

```
#include <string>
```

```
class Book {
```

```
protected:
```

```
    std::string title;
```

```
    std::string author;
```

```
    double price;
```

```
public:
```

```
    void getBookDetails() {
```

```
        std::cout << "Enter Title: ";
```

```
        std::getline(std::cin >> std::ws, title);
```

```

        std::cout << "Enter Author: ";

        std::getline(std::cin >> std::ws, author);

        std::cout << "Enter Price: ";

        std::cin >> price;
    }

    void displayBookDetails() {
        std::cout << "Title: " << title << "\nAuthor: " << author << "\nPrice: " << price << std::endl;
    }
};

class Textbook : public Book {
private:
    std::string subject;

public:
    void getTextbookDetails() {
        getBookDetails();

        std::cout << "Enter Subject: ";

        std::getline(std::cin >> std::ws, subject);
    }

    void displayDetails() {
        displayBookDetails();

        std::cout << "Subject: " << subject << std::endl;
    }
};

int main() {
    Textbook myTextbook;

```



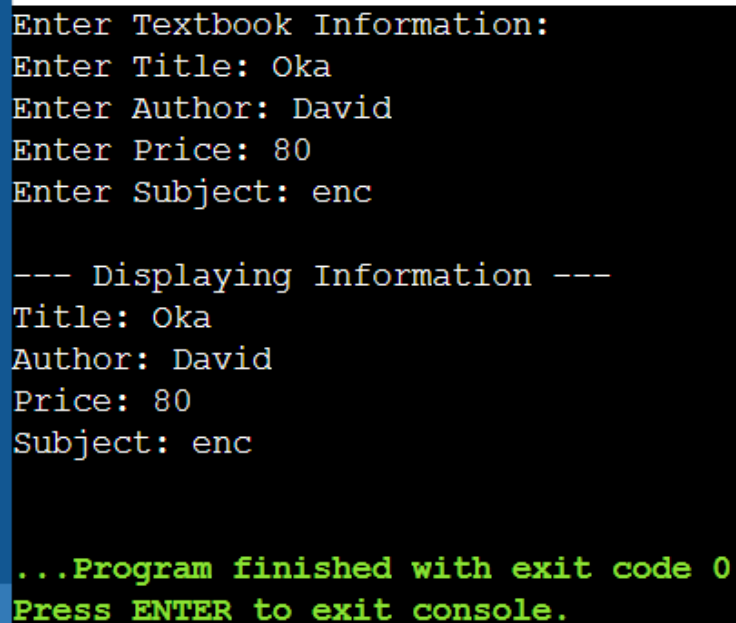
```
std::cout << "Enter Textbook Information:" << std::endl;

myTextbook.getTextbookDetails();

std::cout << "\n--- Displaying Information ---" << std::endl;

myTextbook.displayDetails();

return 0;
}
```



```
Enter Textbook Information:
Enter Title: Oka
Enter Author: David
Enter Price: 80
Enter Subject: enc

--- Displaying Information ---
Title: Oka
Author: David
Price: 80
Subject: enc

...Program finished with exit code 0
Press ENTER to exit console.
```

6. A software company is creating a program to simulate a car's dashboard. The dashboard needs to display speed, fuel level, and temperature. The speed is controlled by a Speedometer class, the fuel level by a FuelGauge class, and the temperature by a Thermometer class. Implement the three classes: Speedometer, FuelGauge, and Thermometer, each with relevant attributes and methods. Then, create a CarDashboard class that inherits from all three classes to display the combined information on the dashboard. Demonstrate how multiple inheritance is used to build this class.

```
#include <iostream>
```

```
class Speedometer {
protected:
    int speed;

public:
    void getSpeed() {
        std::cout << "Enter current speed (km/h): ";
        std::cin >> speed;
    }
};
```

```
class FuelGauge {
protected:
    int fuelLevel;

public:
    void getFuel() {
        std::cout << "Enter fuel level (%): ";
        std::cin >> fuelLevel;
    }
};
```

```
class Thermometer {
protected:
    int temperature;

public:
    void getTemp() {
        std::cout << "Enter engine temperature (Celsius): ";
        std::cin >> temperature;
    }
};
```

```

class CarDashboard : public Speedometer, public FuelGauge, public Thermometer {
public:
    void display() {
        std::cout << "\n--- Car Dashboard ---" << std::endl;
        std::cout << "Speed:      " << speed << " km/h" << std::endl;
        std::cout << "Fuel Level: " << fuelLevel << "%" << std::endl;
        std::cout << "Temperature: " << temperature << " C" << std::endl;
        std::cout << "-----" << std::endl;
    }
};

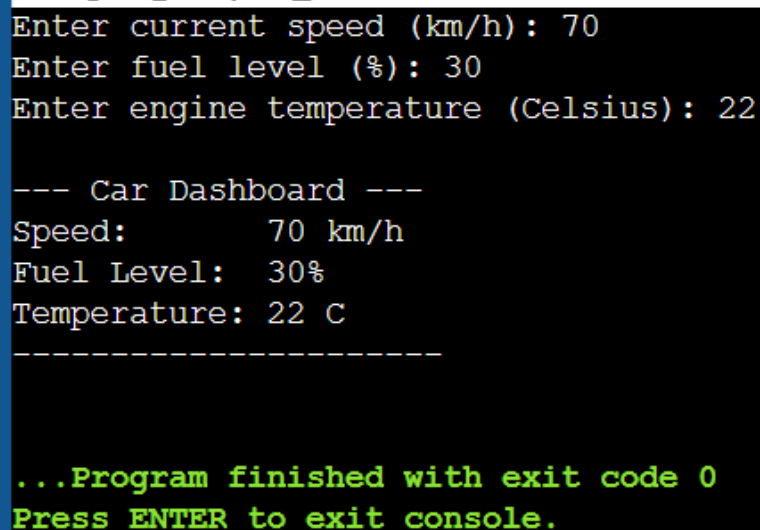
int main() {
    CarDashboard myCar;

    myCar.getSpeed();
    myCar.getFuel();
    myCar.getTemp();

    myCar.display();

    return 0;
}

```



```

Enter current speed (km/h): 70
Enter fuel level (%): 30
Enter engine temperature (Celsius): 22

--- Car Dashboard ---
Speed:      70 km/h
Fuel Level: 30%
Temperature: 22 C
-----

...Program finished with exit code 0
Press ENTER to exit console.

```

7. You are tasked with creating a system for a library that tracks different types of users. The system needs to handle general user information such as name, ID, and contact details. There are two specific types of users: Student and Teacher. Each type of user has additional attributes, such as grade level for students and department for teachers. Implement a base class LibraryUser with general attributes. Then, create two derived classes Student and Teacher that inherit from

LibraryUser and add their own specific attributes. Demonstrate how hierarchical inheritance is applied in this scenario.

```
#include <iostream>
```

```
#include <string>
```

```
class LibraryUser {
```

```
protected:
```

```
    std::string name;
```

```
    int id;
```

```
    std::string contact;
```

```
public:
```

```
    void getGeneralData() {
```

```
        std::cout << "Enter Name: ";
```

```
        std::getline(std::cin >> std::ws, name);
```

```
        std::cout << "Enter ID: ";
```

```
        std::cin >> id;
```

```
        std::cout << "Enter Contact: ";
```

```
        std::getline(std::cin >> std::ws, contact);
```

```
    }
```

```
    void displayGeneralData() {
```

```
        std::cout << "ID: " << id << "\nName: " << name << "\nContact: " << contact <<
std::endl;
```

```
    }
```

```
};
```

```
class Student : public LibraryUser {
```

```
private:
```

```
    int gradeLevel;
```

```
public:
```

```
    void getStudentData() {
```

```
        getGeneralData();
```

```
        std::cout << "Enter Grade Level: ";
```

```
        std::cin >> gradeLevel;
```

```
    }
```

```
    void displayStudent() {
```

```
        std::cout << "\n--- Student Library Card ---" << std::endl;
```

```
        displayGeneralData();
```

```
        std::cout << "Grade Level: " << gradeLevel << std::endl;
```

```
    }
```

```
};
```

```
class Teacher : public LibraryUser {
```

```
private:
```

```
    std::string department;
```

public:

```
void getTeacherData() {  
  
    getGeneralData();  
  
    std::cout << "Enter Department: ";  
  
    std::getline(std::cin >> std::ws, department);  
  
}
```

```
void displayTeacher() {  
  
    std::cout << "\n--- Teacher Library Card ---" << std::endl;  
  
    displayGeneralData();  
  
    std::cout << "Department: " << department << std::endl;  
  
}
```

};

int main() {

Student s;

Teacher t;

std::cout << "Registering a Student:" << std::endl;

s.getStudentData();

std::cout << "\nRegistering a Teacher:" << std::endl;

t.getTeacherData();

```
s.displayStudent();
```

```
t.displayTeacher();
```

```
return 0;
```

```
}
```

```
Registering a Student:
Enter Name: Aaditya
Enter ID: ide21
Enter Contact: Enter Grade Level:
Registering a Teacher:
Enter Name: Enter ID: Enter Contact: Enter Department:
--- Student Library Card ---
ID: 0
Name: Aaditya
Contact:
Grade Level: 73728

--- Teacher Library Card ---
ID: -1669955648
Name:
Contact:
Department:

...Program finished with exit code 0
Press ENTER to exit console.
```